

LeNet 实现 MNIST 图像分类

实验指导

实验概览

实验目的

- 理解卷积神经网络（CNN）的基本原理和结构，特别是 LeNet 的网络架构。
- 掌握使用 LeNet 网络对 MNIST 数据集进行图像分类的方法。
- 学习如何评估卷积神经网络的分类性能，并调整网络参数以优化性能。
- 掌握深度学习模型训练、验证和测试的完整流程。
- 理解超参数调优对模型性能的影响，掌握模型优化的基本方法。

实验要求

- 熟悉 Python 及 PyTorch 深度学习框架。
- 了解卷积神经网络的基本概念、训练过程和评估方法。
- 具备基础的深度学习知识，特别是图像分类任务的相关理论。
- 了解 MNIST 数据集的特点和预处理方法。

提交内容

实验报告（包含实验目的、步骤、结果分析等），采用 PDF 格式。报告应结构清晰、内容详实、语言流畅。

卷积神经网络

卷积神经网络 (CNN) 是一类专门设计用于处理具有网格结构数据（如图像、视频、语音）的深度学习模型，广泛应用于计算机视觉、图像识别、目标检测等场景。

卷积神经网络通过模拟人类视觉系统的处理方式，自动从图像中提取特征，并进行分类、识别等任务。与传统神经网络相比，CNN 在处理图像等**高维数据**时具有更高的效率和准确性。

基本网络结构

1. 卷积层

- 功能：使用卷积核在输入数据上进行滑动窗口计算，提取局部特征
- 特点：每个卷积核专门检测特定类型的特征（如边缘、角点、纹理）
- 参数：卷积核大小、步长、填充

2. 激活函数

- 作用：引入非线性变换，增强网络的表达能力
- 常用函数：ReLU、Sigmoid、Tanh
- 优势：ReLU 计算简单，能有效缓解梯度消失问题

3. 池化层

- 功能：降低特征图的空间尺寸，减少计算量和参数数量
- 类型：
 - 最大池化：取区域内最大值，保留显著特征
 - 平均池化：取区域内平均值，平滑特征
- 效果：增强平移不变性，防止过拟合

4. 全连接层

- 功能：将卷积层和池化层提取的特征映射到输出空间，如分类标签
- 位置：通常位于网络末端，连接所有神经元
- 作用：完成最终的分类或回归任务

5. 输出层

- 分类任务：使用 Softmax 函数输出类别概率分布
- 回归任务：使用线性激活函数输出连续值

优势特点

- 参数共享**: 同一卷积核在不同位置共享权重，大幅减少参数量。
- 局部连接**: 每个神经元只与局部区域连接，符合图像局部相关性。
- 层次化特征**: 浅层提取基础特征，深层组合成复杂特征。
- 平移不变性**: 池化操作使网络对输入的小幅平移不敏感。

LeNet 网络

LeNet 是由 Yann LeCun 等人于 1998 年提出的**卷积神经网络**架构，奠定了现代卷积神经网络“**卷积-池化-全连接**”的核心架构范式，为后续深度学习模型提供了关键设计基础，其适合处理小规模的数据，例如手写数字识别。

核心网络架构

LeNet 其核心架构主要由卷积层、池化层和全连接层交替堆叠构成。

1. 卷积层 (Convolutional Layer)

使用小尺寸卷积核（如 5x5）提取空间特征，如边缘、角点。

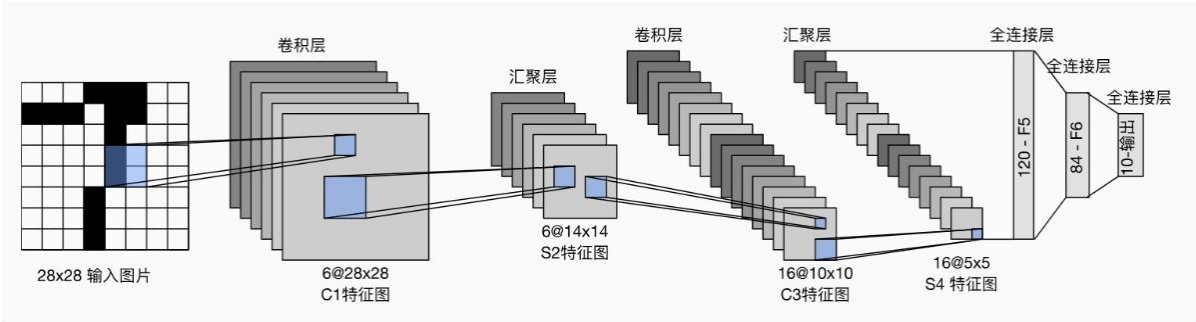
2. 池化层 (Pooling Layer)

采用平均池化或最大池化，对特征图进行下采样，减少数据量并增强特征平移不变性。

3. 全连接层 (Fully Connected Layer)

将最终提取到的二维特征图展平为一维向量，并进行分类输出。

网络结构图

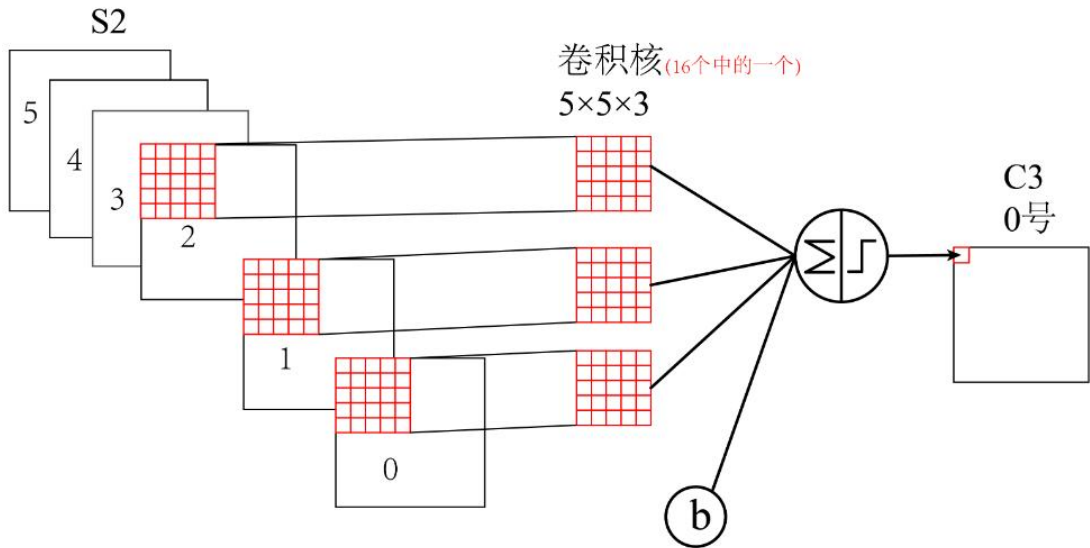


该网络采用“卷积-池化”的交替组合，逐步提取抽象特征并压缩数据维度。

以 28×28 的灰度图像输入；首先经过 C1 卷积层，使用 6 个 5×5 的卷积核生成 6 个 28×28 的特征图；接着通过 S2 池化层降维至 6 个 14×14 的特征图；随后进入 C3 卷积层，利用 16 个卷积核进行特征增强，输出 16 个 10×10 的特征图；再经 S4 池化层压缩为 16 个 5×5 的特征图。最后，特征图被展平并送入多个全连接层进行信息整合，最终由 10 个神经元的输出层完成分类。

不对称连接

在标准的卷积层中，每个输出特征图都是由所有输入特征图通过卷积求和得到的。但 LeNet 的 C3 卷积层并非如此，如下图所示。



以生成 C3 层的第 0 号输出特征图为例，从 S2 层提取 0、1、2 号特征图作为输入，并配属一组独立的 $5 \times 5 \times 3$ 三维卷积核与之对应。卷积操作在这三个指定输入通道上同步进行：卷积核在每个空间位置上与输入进行三维点乘求和，生成

一个初始的二维特征图，随后叠加一个共享的偏置项，并最终通过 tanh 激活函数完成非线性变换，从而得到 C3 层的第 0 号输出特征图。

表 1 C3 与 S2 特征图的对应关系

C3 特征图索引	连接的 S2 特征图索引	连接模式
0	0, 1, 2	连续 3 个
1	1, 2, 3	连续 3 个
2	2, 3, 4	连续 3 个
3	3, 4, 5	连续 3 个
4	0, 4, 5	非连续 3 个
5	0, 1, 5	非连续 3 个
6	0, 1, 2, 3	连续 4 个
7	1, 2, 3, 4	连续 4 个
8	2, 3, 4, 5	连续 4 个
9	0, 3, 4, 5	非连续 4 个
10	0, 1, 4, 5	非连续 4 个
11	0, 1, 2, 5	非连续 4 个
12	0, 2, 3, 4, 5	非连续 5 个
13	0, 1, 3, 4, 5	非连续 5 个
14	0, 1, 2, 4, 5	非连续 5 个
15	0, 1, 2, 3, 4, 5	全部 6 个

以上便是不对称连接的核心机制——每个输出特征图都通过其专属的、与输入通道子集对应的卷积核组来生成，这不仅强制网络学习多样化的特征组合，也显著减少了模型的参数总量。

LeNet 实现 MNIST 数据分类

1. 数据准备与处理

(1) 加载数据集

加载 MNIST 数据集，该数据集包含 70,000 个手写数字图像（60,000 个训练样本，10,000 个测试样本）。其中每个图像为 28×28 像素的灰度图像，分为 10 个类别（0-9）。

(2) 数据预处理

对图像数据进行预处理。

```
# 加载数据集
train_data = datasets.MNIST(root='./data', train=True,
                             transform=transforms.ToTensor(), download=True)

test_data = datasets.MNIST(root='./data', train=False,
                             transform=transforms.ToTensor())

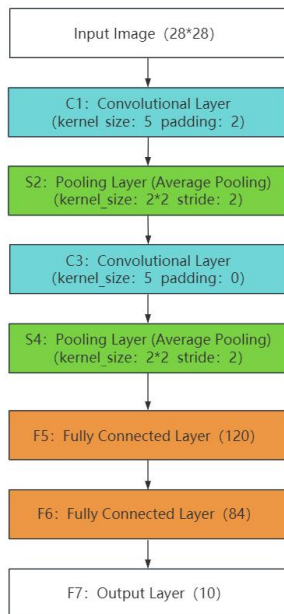
train_gen = torch.utils.data.DataLoader(dataset=train_data,
                                         batch_size=batch_size,
                                         shuffle=True)

test_gen = torch.utils.data.DataLoader(dataset=test_data,
                                       batch_size=batch_size,
                                       shuffle=False)
```

2. 模型构建

(1) 定义 LeNet 网络结构

参考下面的网络结构图，自定义卷积层、池化层和全连接层各层的参数。



```
# 定义LeNet网络
class LeNet(nn.Module): 2 用法
    def __init__(self, num_classes=10):
        super(LeNet, self).__init__()
        # 自定义卷积层、池化层、输出层以及激活函数

    def forward(self, x):
        # 前向传播过程
        # 建议打印各层尺寸，便于后续调试

        return x
```

(2) 定义损失函数和优化器

选择合适的损失函数（交叉熵损失）和优化器，并设置初始学习率。

(3) 训练模型

使用训练集数据实现训练模型，包括前向传播、损失计算、反向传播和参数更新；并保存验证集上性能最好的模型。

3.模型评估与分析

在测试集上评估最终模型的准确率等性能指标。

4. 参数调优

- 尝试调整网络参数，如训练轮数、批处理大小、网络深度、学习率等，以优化分类性能。
- 记录并比较不同参数设置下的实验结果。
- 总结 LeNet 网络结构对实现手写数字分类任务的性能表现。